1) a)

```
Cloning into '/var/lib/cloud9/EDES301'...
remote: Enumerating objects: 24, done.
remote: Counting objects: 100% (24/24), done.
remote: Compressing objects: 100% (16/16), done.
remote: Total 24 (delta 4), reused 3 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (24/24), done.
```

b)

```
debian@beaglebone:/var/lib/cloud9$ cd EDES-301-REAL
debian@beaglebone:/var/lib/cloud9/EDES-301-REAL$ mkdir assignme
mkdir: cannot create directory ‘assignment_03’: File exists
```

c)

```
debian@beaglebone:/var/lib/cloud9/EDES-301-REAL/assignment_0
temp.txt
```

```
  GNU nano 3.2

Hello World!
```

e)

```
Hello World!
```

f)

```
debian@beaglebone:/var/lib/cloud9/EDES-30
temp.txt
debian@beaglebone:/var/lib/cloud9/EDES-30
```

```
debian@beaglebone:/var/lib/cloud9/EDES-
total 12
drwxr-xr-x 2 debian debian 4096 Dec  1
drwxr-xr-x 6 debian debian 4096 Dec  1
-rw-r--r-- 1 debian debian   13 Dec  1
```

2) a)

```
debian@beaglebone:/var/lib/cloud9/EDES-301-REAL/
Filesystem     1K-blocks    Used Available Use%
udev              218884       0    218884   0%
tmpfs              49520     264     49256   1%
/dev/mmcblk0p1   3544568 2268220   1075852  68%
tmpfs             247588       0    247588   0%
tmpfs               5120       0      5120   0%
tmpfs             247588       0    247588   0%
```

b)

```
debian@beaglebone:/var/lib/cloud9/EDES-301-REAL/assignment_03$ sudo /opt/scripts/tools/grow_partition.sh
[sudo] password for debian:
Media: [/dev/mmcblk0]
sfdisk: 2.26.x or greater
Disk /dev/mmcblk0: 29.1 GiB, 31267487744 bytes, 61069312 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0x5ea0f889

Old situation:

Device         Boot Start     End Sectors  Size Id Type
/dev/mmcblk0p1 *     8192 7372799 7364608  3.5G 83 Linux

>>> Created a new DOS disklabel with disk identifier 0xa0a508e4.
/dev/mmcblk0p1: Created a new partition 1 of type 'Linux' and of size 29.1 GiB.
Partition #1 contains a ext4 signature.
/dev/mmcblk0p2: Done.
```

```
Partition #1 contains a ext4 signature.
/dev/mmcblk0p2: Done.

New situation:
Disklabel type: dos
Disk identifier: 0xa0a508e4

Device         Boot Start      End  Sectors  Size Id Type
/dev/mmcblk0p1 *     8192 61069311 61061120 29.1G 83 Linux

The partition table has been altered.
Calling ioctl() to re-read partition table.
Re-reading the partition table failed.: Device or resource busy
The kernel still uses the old table. The new table will be used at the next reboot or after you run partprobe(8) or kpartx(
Syncing disks.
```

3) Everything installed! (No screenshots needed)
4) a) 64,206
   - Binary: 1111101000001110_2
   - Octal: 175016_8
   - Hexadecimal: $FA0E_{16}$

   b) 48,879
   - Binary: 11011111011101111_2
   - Octal:137357_8
   - Hexadecimal: $BEEF_{16}$

   c) 57,005
   - Binary: 1101111011101101_2
   - Octal: 157355_8
   - Hexadecimal: $DEED_{16}$

5) a)
   i) $00101010_2 = 41_{10}$
   ii) $11111111_2 = 255_{10}$
   iii) 2024_{10}
   iv) 44269_{10}
   b)
   i) $00101010_2 = 42_{10}$
   ii) $11111111_2 = -1_{10}$
   iii) 2025_{10}
   iv) -21267_{10}
   c)
   i) 0x49 = 73_{10} → ‘I’
   ii) ‘I’,’o’,’v’,’e’ → ‘love’

iii) ‘E’,’D’,’E’,’S’ → ‘EDES’
iv) $3{,}354{,}673_{10}$ = 0x334531
0x33 → ‘3’
0x45 → ‘E’
0x31 → ‘1’
→ ‘3E1’

6) a) The rule I have the most trouble agreeing with is the 79-character line limit. I get that it comes from the days of small terminals and side-by-side diffs, and the guide even acknowledges that, but in practice it feels outdated. Most people today are coding on widescreen monitors or at least a modern laptop, so there's no real reason a line of code has to wrap at 79 charaters. Sometimes you're writing something like a long function call with multiple arguments and breaking it across several lines just makes it harder to read. The guide says teams can agree to go up to 99 or even 119 characters, which honestly should just be the default at this point, because wrapping code to hit an arbitrary number doesn't automatically make it more readable, sometimes it actually makes you lose track of the logic because everything is split up. I think the spirit of not wanting people to write endlessly long lines is valid, but 79 seems too little for the demand we have nowadays.

b) Yes, I do agree that whitespace should matter in Python, even though it was a little annoying when I first started learning Python. With my first languages being C/C++, it felt weird that indentation wasn't just a style choice but an actual part of the syntax. But after writing more code I think it's one of the best things about Python, because in languages where indentation is optional, people will still do it since unindented code is nearly unreadable. Python just makes it mandatory, which means everyone's code (usually) ends up looking cleaner. You open someone else's file and you can follow it without having to try to make sense of that person’s syntax. That said, the guide's rules around blank lines between functions and around operators might be somewhat exaggerated, as I don’t think it affects readability as much.

c) Something I found interesting was the section on naming conventions, specifically the warning about never using lowercase l, uppercase O, or uppercase I as single-character variable names. The reason is that depending on the font, they're basically indistinguishable from the numbers 1 and 0. It's a small detail but it's the kind of thing that could genuinely waste an hour of debugging time before you even realize what's wrong. It made me think about how many style rules exist not because someone decided it would look cleaner, but because someone actually ran into a frustrating bug because of it. That context makes the guide feel a lot more practical than just a list of formatting preferences.